

ASSIGNMENT

Employee Attendance Portal

Duration	3 Days
Frontend	React.js
Backend	Node.js
Database	PostgreSQL

1. Overview

You are required to build a web-based Employee Attendance Portal for a company. The system must support multiple user roles — Employee, Manager, and HR — each with distinct capabilities and access levels. The application should handle attendance logging, leave management with an approval workflow, and role-based permission control throughout. The focus of this assignment is on building a well-architected, secure fullstack application that reflects real-world business logic.

2. User Roles & Management

The system must support three distinct roles. Each role has its own login and is restricted to actions appropriate to their level of access.

2.1 Employee

A regular staff member. They can mark their own attendance, view their own timesheet, apply for leave, and track the status of their leave requests. They cannot view other employees' data or take any action on leave requests.

2.2 Manager

A team lead or senior staff member responsible for one or more employees. In addition to all employee capabilities, a manager can view the attendance records of employees under them, and act on leave requests submitted by their reportees — approving or rejecting with a remark. A manager cannot manage other managers or access HR-level functions.

2.3 HR

An HR administrator with the broadest access. HR can view attendance data across the entire organization, manage user accounts (create, deactivate, assign roles, assign reporting manager), configure leave types, and view all leave requests regardless of approval status. HR does not participate in the leave approval workflow — that responsibility lies with the Manager.

3. Tech Stack

You must use the following technologies. Any deviation from this stack will be considered incomplete.

3.1 Frontend — React.js

Build the UI using React.js with client-side routing. You may use any component library of your choice. Role-based rendering must be handled on the frontend — menus, pages, and actions should be visible only to users who have permission to perform them.

3.2 Backend — Node.js

Build the API using Node.js following RESTful conventions. Every protected route must validate the session token and verify that the requesting user has the appropriate role and permissions before processing the request. Role checks must be enforced server-side — frontend-only restrictions are not acceptable.

3.3 Database — PostgreSQL

Use PostgreSQL as the database. Use Sequelize as the ORM. Your schema should reflect the relationships between users, roles, teams, attendance records, and leave requests. Include migration files in the submission.

3.4 Authentication & Session Management

Implement login using username and password. On successful login the server must issue a session token. The token must expire after 15 minutes of inactivity — this is a hard requirement. Expired or missing tokens must be rejected server-side and the frontend must redirect to the login page.

4. Permission Management

Access control must be enforced at the API level. The table below defines which actions are permitted for each role. Attempting a restricted action should return an appropriate HTTP error.

Action	Employee	Manager	HR
View own attendance	✓	✓	✓
Check in / Check out	✓	✓	✗
Apply for leave	✓	✓	✗
View own leave requests	✓	✓	✓
View team attendance	✗	✓	✓
Approve / Reject leave	✗	✓	✗
View all leave requests	✗	✗	✓
Create / Deactivate users	✗	✗	✓
Assign roles & managers	✗	✗	✓

Configure leave types	X	X	✓
View organization attendance	X	X	✓

5. Application Pages

The application must include the following pages. Pages and actions should render based on the logged-in user's role.

5.1 Login Page

A single login page for all roles. The system should identify the user's role post-authentication and redirect them to the appropriate landing page.

5.2 Check-in / Check-out Page

Available to Employees and Managers. The user can mark their check-in and check-out for the day. Current status for the day should be clearly shown. An employee or manager cannot check in more than once per calendar day.

5.3 Timesheet Page

Available to all roles. Employees and Managers see their own attendance history with date, check-in time, check-out time, and total hours worked. Managers additionally have a view to see attendance records for employees reporting to them.

5.4 Apply Leave Page

Available to Employees and Managers. The user fills a form with leave type, start date, end date, and reason, and submits the request. The page also lists all past leave requests along with their current status — Pending, Approved, or Rejected — and the remark provided by the manager on actioned requests.

5.5 Leave Approval Page

Available to Managers only. Displays all pending leave requests submitted by employees under the manager. The manager can open each request, review the details, and either approve or reject it with a mandatory remark. Once acted upon, the status of the request is updated and the employee can see the outcome on their Apply Leave page.

5.6 User Management Page

Available to HR only. Displays a list of all users in the system. HR can create new user accounts, assign them a role (Employee or Manager), and assign them a reporting manager. HR can also deactivate accounts. Deactivated users should not be able to log in.

5.7 Leave Configuration Page

Available to HR only. HR can define and manage the types of leave available in the system (e.g. Casual Leave, Sick Leave, Earned Leave). Each leave type should have a name and an annual quota. Changes to leave types should reflect across the system.

6. Leave Approval Workflow

The leave management process follows a maker-checker pattern. The sequence is as follows:

- The employee submits a leave request. The request is created in a Pending state.
- The request appears in the Leave Approval page of the employee's assigned manager.
- The manager reviews the request and either approves or rejects it, providing a remark in both cases.
- The status of the request is updated to Approved or Rejected. The remark is stored and visible to the employee.
- An employee cannot cancel or modify a request once it has been acted upon by the manager.
- A manager cannot approve or reject their own leave requests. Their requests follow the same workflow and must be handled by HR or a designated authority.

7. Constraints & Assumptions

- Session tokens must expire after 15 minutes of inactivity. This is a hard requirement.
- All permission checks must be enforced server-side. Frontend-only access control is not sufficient.
- An employee or manager can only check in once per calendar day.
- A manager cannot act on their own leave requests.

8. Submission

Submit the following within 3 days of receiving this document.

- README.md — Setup instructions, environment variables, how to create the database, run migrations, and start both servers.
- frontend.zip — Complete React.js frontend source code.
- backend.zip — Complete Node.js backend source code including all Sequelize migration files.
- database.sql — A SQL script that creates all required tables and seeds the initial data needed to run the application, including at least one user of each role.